

4.3 Default Initialization

The duration of a variable (whether static or automatic) also affects its default initialization.

If you do not explicitly initialize them, automatic-duration variables (that is, local, non-`static` ones) are not guaranteed to have any particular initial value; they will typically contain garbage. It is therefore a fairly serious error to attempt to use the value of an automatic variable which has never been initialized or assigned to: the program will either work incorrectly, or the garbage value may just happen to be ``correct" such that the program appears to work correctly! However, the particular value that the garbage takes on can vary depending literally on *anything*: other parts of the program, which compiler was used, which hardware or operating system the program is running on, the time of day, the phase of the moon. (Okay, maybe the phase of the moon is a bit of an exaggeration.) So you hardly want to say that a program which uses an uninitialized variable ``works"; it may seem to work, but it works for the wrong reason, and it may stop working tomorrow.

Static-duration variables (global and `static` local), on the other hand, are guaranteed to be initialized to 0 if you do not use an explicit initializer in the definition.

(Once upon a time, there was another distinction between the initialization of automatic vs. static variables: you could initialize *aggregate* objects, such as arrays, only if they had static duration. If your compiler complains when you try to initialize a local array, it's probably an old, pre-ANSI compiler. Modern, ANSI-compatible compilers remove this limitation, so it's no longer much of a concern.)

Read sequentially: [prev](#) [next](#) [up](#) [top](#)

This page by [Steve Summit](#) // [Copyright](#) 1995-1997 // [mail feedback](#)